

SCOS Real-Time — Project Context

For Claude Code. Compact reference for the SCOS real-time rewrite. Full plan: [docs/Full_Plan_RealTime.pdf](#). Protocol spec: [docs/SCOS_protocol.md](#).

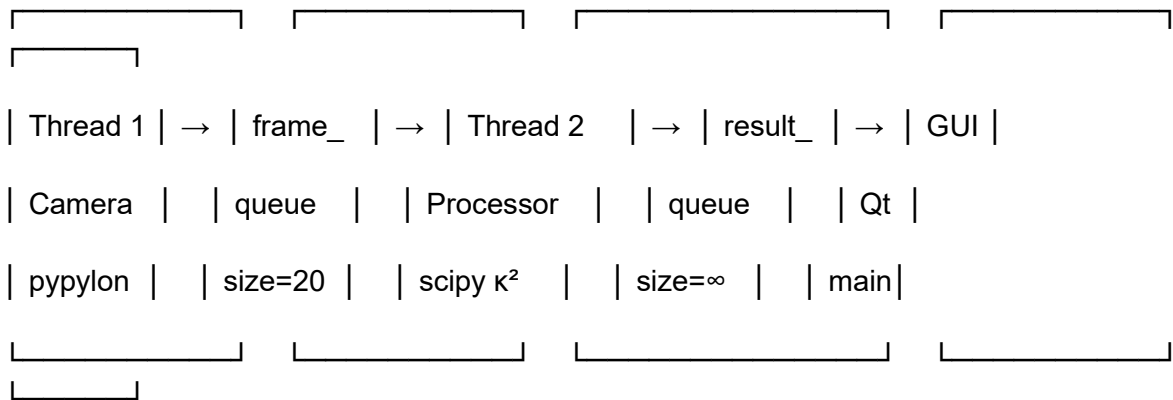
What this project is

Speckle Contrast Optical Spectroscopy (SCOS) for cerebral blood flow. We are **rewriting the existing offline Python code as a real-time GUI application**. Lab: Optical Neuroimaging Lab, Bar-Ilan University.

System parameters

Parameter	Value
Frame size	700 × 700 pixels, Mono12 in uint16 (~1 MB/frame)
Frame rate	~20 Hz (target; possibly 40 Hz if budget allows)
Recording length	up to several hours (default 5 min, max 4 h)
Camera	Basler GigE via pypylon
Per-frame compute budget	1000/20 = 50 ms

Architecture: 3 threads + 2 queues (producer–consumer)



~20 Hz FIFO blocking reads timeout 0.5s ProcessingResult QTimer 100ms

- **Thread 1 (Camera):** `QThread/threading.Thread`. Loop calls `camera.RetrieveResult()` and `frame_queue.put(frame)`. **Never silently drops frames.** If queue 80% full → pause capture + raise warning (see "Overload behavior").
 - **Thread 2 (Processor):** Reads `frame_queue.get(timeout=0.5)`, runs `compute_kappa2()`, pushes `ProcessingResult` to `result_queue`. Reacts to state transitions.
 - **Thread 3 (GUI, main):** Never blocks. `QTimer.timeout` every 100 ms drains `result_queue` non-blocking, updates plot. Cross-thread comms via `pyqtSignal` only.
-

Critical decisions (already settled — do not change without discussion)

Pylon capture strategy

- Use `pylon.GrabStrategy_OneByOne` (NOT `LatestImageOnly`).
- Set `camera.MaxNumBuffer.Value = 20`.
- **Why:** `LatestImageOnly` silently drops old frames → breaks FFT (uneven sampling) → wrong pulse rate, no error. SCOS feeds the $1/\kappa^2$ time series into FFT for heart rate; missed frames are a silent scientific bug.
- **Detect drops** via `result.GetNumberOfSkippedImages()` per grab.

`frame_queue` size = 20 (all states: preview, calibration, measurement)

- Memory cost: $20 \times 1 \text{ MB} = 20 \text{ MB}$ (negligible).
- High-water mark at **80%** = 16 frames = ~800 ms of user reaction time at 20 Hz.
- Smaller queues (e.g. 2) give only ~80 ms — too short to read a dialog.

Overload behavior — never drop silently

When `frame_queue` fill $\geq 80\%$:

1. **Pause capture** (don't stop, don't reset — preserves calibration state).
2. Show **non-blocking dialog** with 4 options (least → most disruptive):
 - Reduce ROI (best first choice; fewer pixels → instant speedup)
 - Reduce FPS (changes temporal resolution → affects FFT/BPM accuracy)
 - Save-only mode (write raw frames, skip κ^2 ; process offline later)

- Process-only mode (skip disk write, keep computing κ^2 ; risky)
- 3. After **5–10 s timeout** → default to "save-only" mode.
- 4. Always show two GUI indicators: queue-fill bar (0–100%) + dropped-frame counter.

Three buffering layers (do not confuse)

1. Pylon SDK buffer (controlled by `MaxNumBuffer` + `GrabStrategy`)
2. Python `frame_queue` (controlled by `queue.Queue(maxsize=...)`)
3. Currently-processing frame (no control, just CPU time)

Bigger buffers do **not** fix a slow processor — they only delay frame loss. Real fixes: faster math, smaller ROI, lower FPS.

Target folder structure

scos-python/

```

├── core/                # no GUI, no Qt
|   ├── scos_math.py    # pure functions:  $\kappa^2$ , noise correction
|   ├── frame_source.py # ABC: FrameSource
|   ├── camera_source.py # CameraFrameSource (Basler/pypylon)
|   ├── folder_source.py # FolderFrameSource (test/dev w/o camera)
|   ├── session.py      # SCOSSession + state machine
|   ├── pipeline.py     # RealtimePipeline (threads + queues)
|   └── recorder.py     # progressive disk save
├── gui/
|   ├── main_window.py
|   ├── image_widget.py
|   ├── plot_widget.py
|   └── controls_panel.py

```

```
|   └─ status_bar.py
|
|─── arduino_uploader.py    # kept as-is
|
|─── tests/
|
|   └─ data/
|
|   |   └─ offline_reference.npz  # ground truth from old offline code
|   |
|   |   └─ sample_frames.npz     # first 20 frames of reference recording
|   |
|   |   └─ golden_frames.npz
|   |
|   └─ test_scos_math.py
|
|   └─ test_frame_source.py
|
|   └─ test_session.py
|
|   └─ test_pipeline.py
|
|─── docs/
|
|   └─ SCOS_protocol.md        # existing
|
|   └─ realtime_architecture.md  # to be written in Phase 1
|
|─── pyproject.toml
|
|─── requirements.lock
|
|─── main.py
```

Key data structures

```
# core/session.py
```

```
from enum import Enum, auto
```

```
class State(Enum):
```

```

IDLE = auto()

DARK_CAL = auto()      # capture N1 frames, laser OFF

BRIGHT_CAL = auto()    # capture N2 frames, laser ON, no subject

MEASURING_INIT = auto() # first norm_seconds — gather BFI for normalization

MEASURING = auto()     # main loop, plot rBFI

FINISHED = auto()

ERROR = auto()

```

```
@dataclass
```

```
class ProcessingResult:
```

```

    timestamp: float      # seconds since measurement start

    kappa2_raw: float

    kappa2_fixed: float   # noise-corrected

    bfi: float            # = 1 / kappa2_fixed

    rbfi: float | None    # normalized; None until mean_BFI_first known

    mean_intensity: float # <I> inside ROI

    frame_number: int

```

```
@dataclass
```

```
class SessionConfig:
```

```

    window_size: int = 7      # spatial filter window (odd)

    n_dark_frames: int = 600   # N1

    n_bright_frames: int = 600 # N2

    recording_minutes: float = 5.0 # inf = unbounded (max 4 h)

```

```
norm_seconds: float = 5.0    # window for mean_BFI_first

save_frames: bool = False

output_folder: str = ""

G_due: float = 0.0          # from camera-SN + pixel_format table
```

Core math ([core/scos_math.py](#))

Pure functions, no I/O, no state, no Qt.

```
#  $\kappa^2_{\text{fixed}} = (\text{var\_raw} - \text{var\_dark} - \text{var\_bright} - G \cdot \langle I \rangle - 1/12) / \langle I \rangle^2$ 
```

```
convert_gain_db_to_due(gain_db, bit_depth, sat_capacity_e) -> float
```

```
#  $G_0 = (2^{\text{bit\_depth}}) / \text{sat\_capacity\_e}$ 
```

```
# returns  $G_0 * 10^{(\text{gain\_db} / 20)}$ 
```

```
# Basler ace sat_capacity_e  $\approx 10500$ 
```

```
local_mean_and_variance(im, window) -> (mean_im, var_im)
```

```
# uses scipy.ndimage.uniform_filter; clamps var  $\geq 0$ 
```

```
shrink_mask_for_window(mask, window) -> np.ndarray
```

```
# binary_erosion by  $\lceil \text{window}/2 \rceil + 1$  — keeps filter window inside ROI
```

```
compute_kappa2(frame, mask, window, mean_dark,
```

```
    var_dark_filtered, var_bright_filtered, G_due)
```

```
-> (kappa2_raw, kappa2_fixed, mean_intensity)
```

Validation rule: [test_offline_reference_matches](#) must pass — new math must reproduce old offline output to [rel=1e-6](#) on the 20 sample frames (with zero calibration arrays, since reference has no calibration).

What stays from old code

- `camera.py` — pypylon wrapper, use as-is or with minor edits
- κ^2 formulas in `processor.py` — copy into clean `core/scos_math.py`
- `convert_gain` — extend to load G[DU/e] from a table by camera SN + pixel format
- `tests/` — extend, don't replace
- `scos_config.json` — extend with new params (`window_size`, N1, N2, `recording_length`, etc.)

Gap map — what's NOT yet implemented

G[DU/e] lookup from table (currently hardcoded 10500)

Dark calibration N1=600 with proper spatial filter

Bright calibration N2=600 with spatial filter

`var_bright` spatial filter

ROI shrink by `window/2 + 1`

rBFI normalization on first N seconds

Progressive disk save (HDF5 append every M minutes)

State machine (DARK → BRIGHT → MEASURING)

Pop-up dialogs ("turn off laser", "turn on laser, remove subject")

Overload dialog (4 options)

Queue-fill + dropped-frame indicators

Long-recording considerations (4 h × 20 Hz = 288 000 frames)

Issue	Decision
Saving all frames = ~280 GB	Off by default. If on: save only N1+N2 calibration frames (~1.2 GB) + every K-th frame (configurable subsample). Check free disk space; stop below threshold.
Plot with 200k+ points	Keep full data in <code>np.array</code> (~7 MB for κ^2 /BFI/rBFI arrays). Display via <code>PlotDataItem.setDownsampling(auto=True)</code> → max 5000 visible points.

Issue	Decision
$rBFI = BFI / \text{mean_BFI_firstSeconds}$ over hours	Open question for supervisor. Laser drift / detector heating / subject motion may invalidate fixed normalization. Options: rolling re-normalization, or save raw BFI and normalize offline.
Crash after 3.5 h loses everything	Progressive save every M min (default 5) to HDF5 (supports <code>append</code> ; <code>.mat</code> does not). Final flush in <code>closeEvent</code> .
Connection stability	Use <code>logging</code> module everywhere. On camera disconnect → reconnect attempt, don't crash. Save accumulated data on any error. GUI health indicator: camera status, dropped-frame count, timing stats.

GUI rules

- Main thread: **never** `while True`, `sleep`, or heavy compute.
- Plot updates: **buffered**. `_on_result_ready` appends to `self._pending_results`; `QTimer @ 100 ms` drains the buffer to the plot.
- Image display: every **2.5 s** during **MEASURING** (per protocol); 30 FPS during preview/IDLE.
- Stop during calibration → confirmation dialog → **FINISHED** state, save partial data.

Phase plan (do in order)

Phase	Duration	Goal
0. Reference baseline	1 day	Run old offline code on one good 1–5 min recording → save as <code>tests/data/offline_reference.npyz</code> and <code>sample_frames.npyz</code> (first 20).

Phase	Duration	Goal
1. Architecture on paper	1 day	Write <code>docs/realtime_architecture.md</code> . Answer all open design questions before coding.
2. Tooling lock-in	½ day	<code>pyproject.toml</code> , <code>pip-compile</code> → <code>requirements.lock</code> , <code>ruff</code> , <code>mypy</code> , <code>pre-commit</code> .
3. Code, bottom-up	2–4 weeks	Layer 1 math → 2 frame source → 3 session → 4 pipeline → 5 GUI. Each layer fully tested before next.
4. Polish	ongoing	GitHub Actions CI, <code>pytest-qt</code> , <code>pytest-benchmark</code> , README, replace <code>print()</code> → <code>logging</code> , <code>.claude/settings.json</code> hooks.

Tooling baseline

- Python ≥ 3.11
- Deps: `pypylon`, `PyQt6`, `pyqtgraph`, `numpy`, `scipy`, `opencv-python`, `pyserial`, `tifffile`, `h5py`
- Dev: `pytest`, `pytest-cov`, `pytest-qt`, `hypothesis`, `ruff`, `mypy`, `pre-commit`, `pip-tools`
- `ruff`: line-length 100, target py311, select `["E", "F", "I", "B", "UP", "SIM", "PERF", "NPY", "RUF"]`
- `mypy`: strict, files = `["core/"]`

Coding principles

1. **GUI is the LAST layer, not the first.**

2. **Never silently drop frames.**
3. Heavy work never on the main thread.
4. Threads communicate via queues + `pyqtSignal` only — no shared variables.
5. Tests for layer N before starting layer N+1.
6. Small commits; every commit must build + pass fast tests.
7. Profile before optimizing.
8. `logging` everywhere, never `print()`.
9. When in doubt about architecture: write on paper first, then ask.

Open questions (supervisor input needed)

- rBFI normalization strategy on multi-hour recordings
- Disk-space limit policy
- Where to store the per-X-seconds plot/image refresh interval (config file vs menu)